IThub

Массивы, циклы.
Методы, параметры методов.



IThub

Цикл for

управляющая конструкция программного кода, которая нарушает линейность исполнения алгоритма и позволяет выполнить указанный код множество раз.

```
for (инициализация; условие; итерация) {
    // тело цикла
}
```

бесконечный цикл for

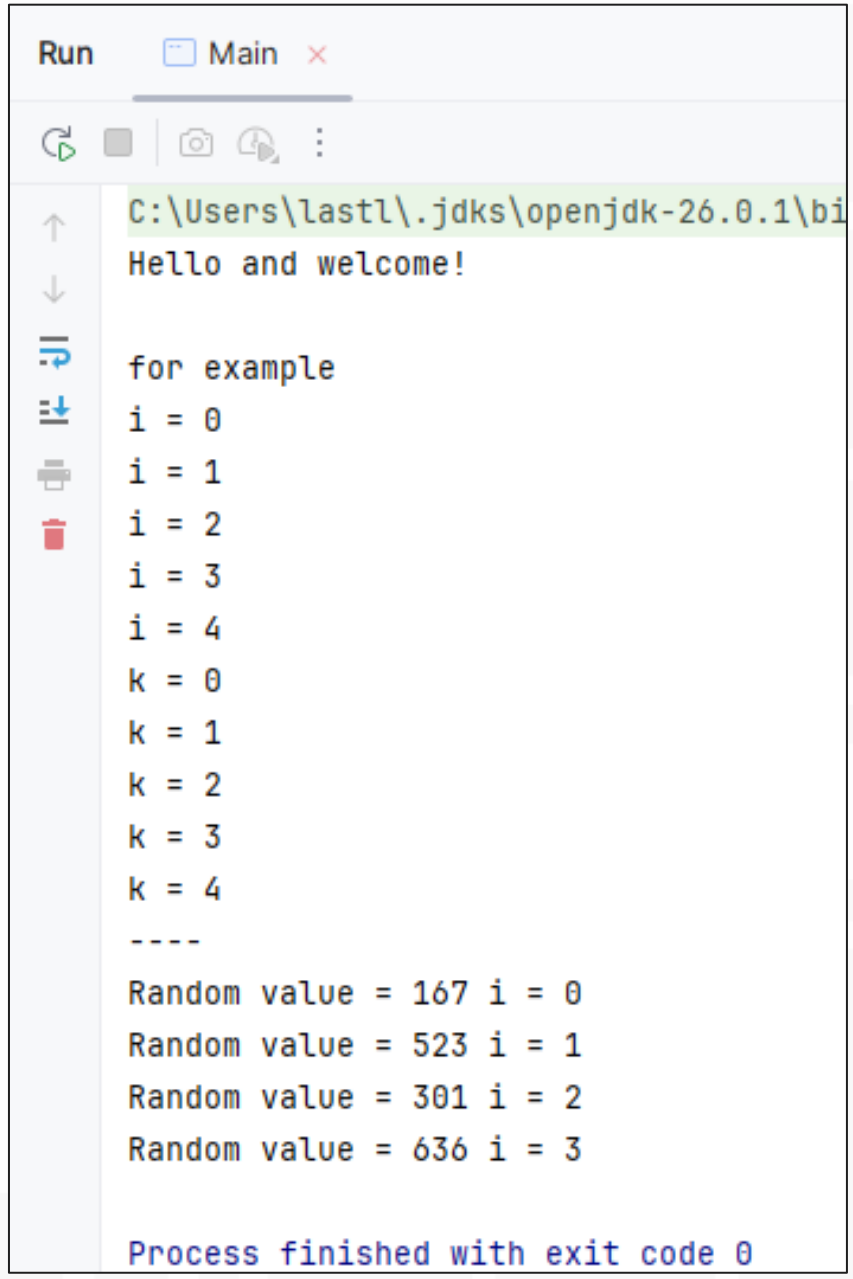
```
for ( ; ; ) {
    // …
{
```

```java
public void forOperatorExample() {
    System.out.println("for example");
    int i;
    for (i = 0; i < 5; i++) {
        System.out.println("i = " + i);
    }

    // объявление переменной k внутри цикла for
    for (int k = 0; k < 5; k++) {
        System.out.println("k = " + k);
    }
}
```

```java
public void forOperatorExample2() {
    int randomValue;
    int mod = -1;

    // условием может быть любое булевское выражение
    for (int i = 0; !(mod == 0); i++) {
        randomValue = new Random().nextInt(1000);
        mod = randomValue % 2;
        System.out.println("Random value = " + randomValue + " i = " + i);
    }
}
```

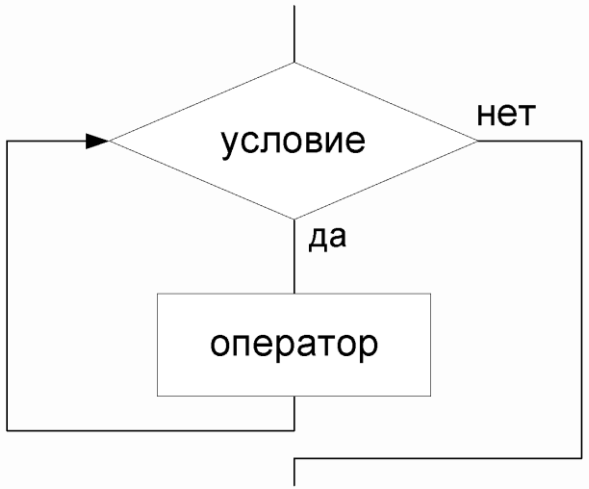


IThub

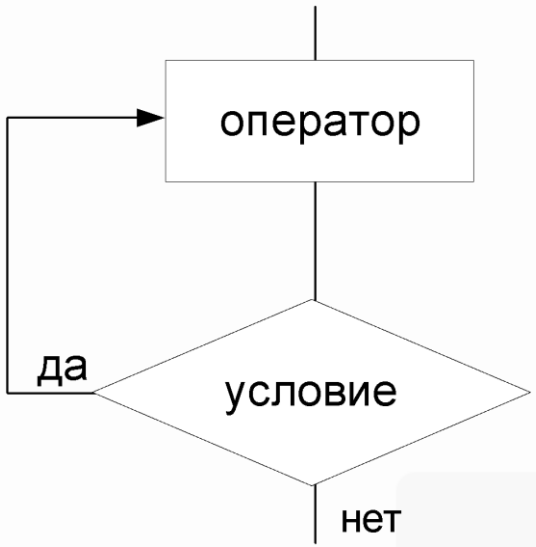
Цикл while

- повторяет выполнение оператора или блока до тех пор, пока истинно управляющее выражение

```
while (условие) {
    // тело цикла
}
```




```
do {
    // тело цикла
} while (условие);
```



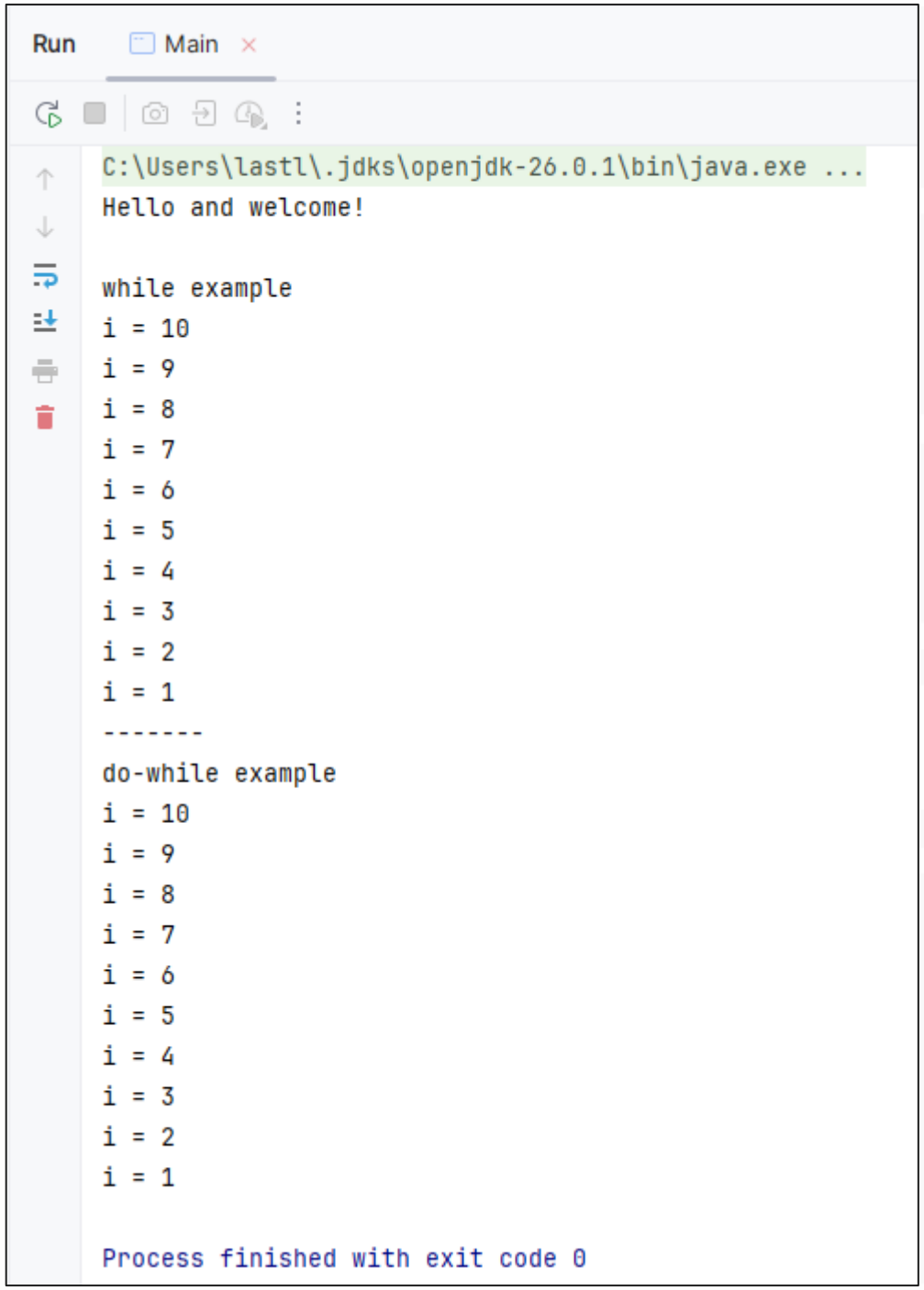
бесконечный цикл while

```
while (true) {
    // …
{
```

IThub

```java
public void whileOperatorExample() {
    System.out.println("while example");
    int i = 10;
    while (i > 0) {
        System.out.println("i = " + i);
        i--;
    }
}

public void doWhileOperatorExample() {
    System.out.println("do-while example");
    int i = 10;
    do {
        System.out.println("i = " + i);
        i--;
    } while (i > 0);
}
```



IThub

Массивы

Массив - это группа переменных одного типа, к которой можно обращаться по общему имени. Доступ к определенному элементу массива осуществляется по его индексу.

Одномерные массивы

- общая форма объявления одномерного массива

  тип [ ] имя_переменной;

- массивы могут инициализироваться при их объявлении
- при подобной инициализации все элементы массива имеют значение по умолчанию. Для числовых типов (в том числе для типа char) это число 0, для типа boolean это значение false, а для остальных объектов это значение null.

```java
public class ArraysExample {  3 usages  Aleksandra Zakharova *

    public ArraysExample() {  1 usage  Aleksandra Zakharova
    }

    public void initArrays() {  1 usage  Aleksandra Zakharova
        int[] numbers; // объявления одномерного массива

        numbers = new int[10]; // инициализация массива
        numbers[0] = 0;
        numbers[1] = 1;
        numbers[2] = 2;
        numbers[3] = 3;
        numbers[4] = 4;
        numbers[5] = 5;
        numbers[6] = 6;
        numbers[7] = 7;
        numbers[8] = 8;
        numbers[9] = 9;

        System.out.println("the biggest number = " + numbers[9]);
    }
```

Run Main

```
C:\Users\lastl\.jdks\openjdk-26.0.1\bin\java.exe ...
Hello and welcome!

the biggest number = 9

Process finished with exit code 0
```

```java
        System.out.println("the biggest number = " + numbers[9]);

        char[] letters = { 'a', 'b', 'c', 'd', 'e', 'f' };
        System.out.println("First letter is '" + letters[0] + "'");
    }

    public void initMdArray() {  no usages  Aleksandra Zakharova
```

Run Main

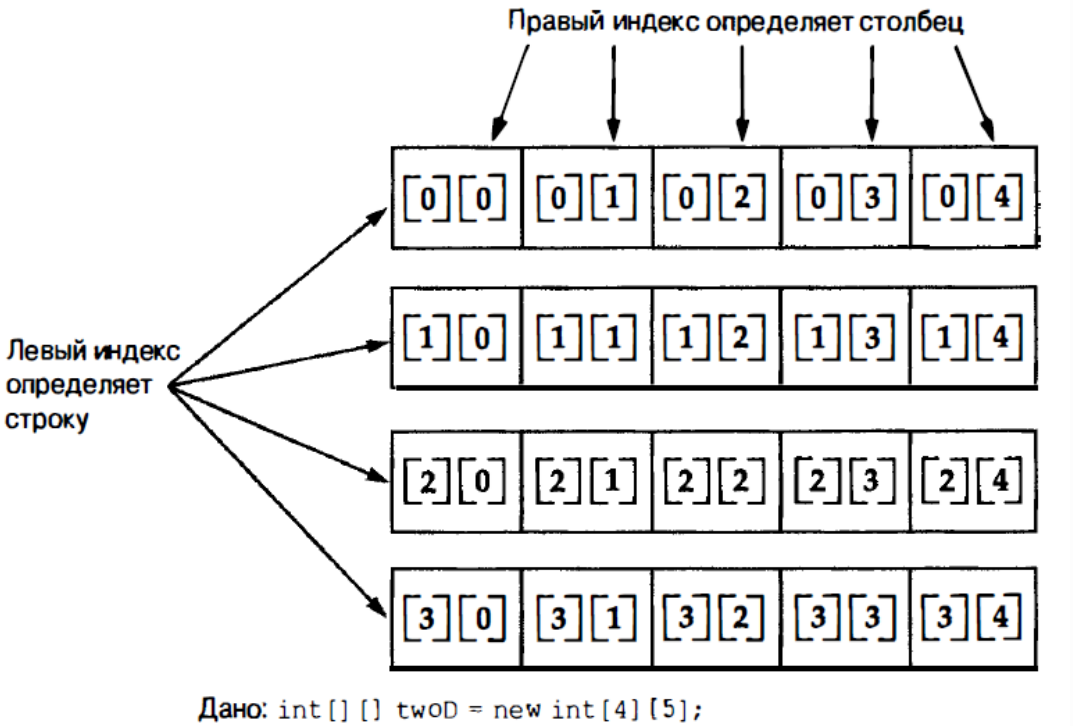
```
C:\Users\lastl\.jdks\openjdk-26.0.1\bin\java.exe ...
Hello and welcome!

the biggest number = 9
First letter is 'a'

Process finished with exit code 0
```

МНОГОМЕРНЫЕ МАССИВЫ

- Многомерные массивы в Java реализованы как массивы массивов



Дано: `int[][] twoD = new int[4][5];`

```java
public void initMdArray() {
    int[][] twoD = new int[4][5];

    int i, j, k = 0;
    for (i = 0; i < 4; i++) {
        for (j = 0; j < 5; j++) {
            twoD[i][j] = k;
            k++;
        }
    }

    for (i = 0; i < 4; i++) {
        for (j = 0; j < 5; j++)
            System.out.print(twoD[i][j] + " ");
        System.out.println();
    }
}
```

```
C:\Users\lastl\.jdks\openjdk-26.0.1\bin\
Hello and welcome!

0 1 2 3 4
5 6 7 8 9
10 11 12 13 14
15 16 17 18 19

Process finished with exit code 0
```

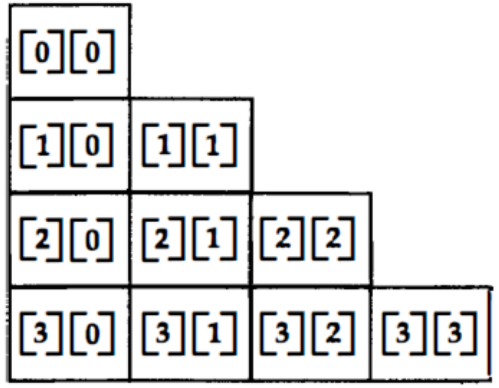
IThub

```java
public void initMdArray2() {
    int[][] twoD = new int[4][5];

    twoD[0] = new int[1];
    twoD[1] = new int[2];
    twoD[2] = new int[3];
    twoD[3] = new int[4];
    int k = 0;
    for (int i = 0; i<4; i++) {
        for (int j = 0; j<i+1; j++) {
            twoD[i][j] = k;
            k++;
        }
    }

    for (int i=0; i<4; i++) {
        for (int j = 0; j<i+1; j++)
            System.out.print(twoD[i][j] + " ");
        System.out.println( );
    }
}
```

- поскольку многомерные массивы являются массивами массивов, длина каждого массива находится под вашим контролем




```java
int [][] theSameArray = {
        {0},
        {1, 2},
        {3, 4, 5},
        {6, 7, 8, 9},
    };
```

```
C:\Users\lastl\.jdks\openjdk-26.0.1\bin\java.exe ...
Hello and welcome!

0
1 2
3 4 5
6 7 8 9

Process finished with exit code 0
```

размеры массивов во втором измерении не равны

Внимательно копируем массивы

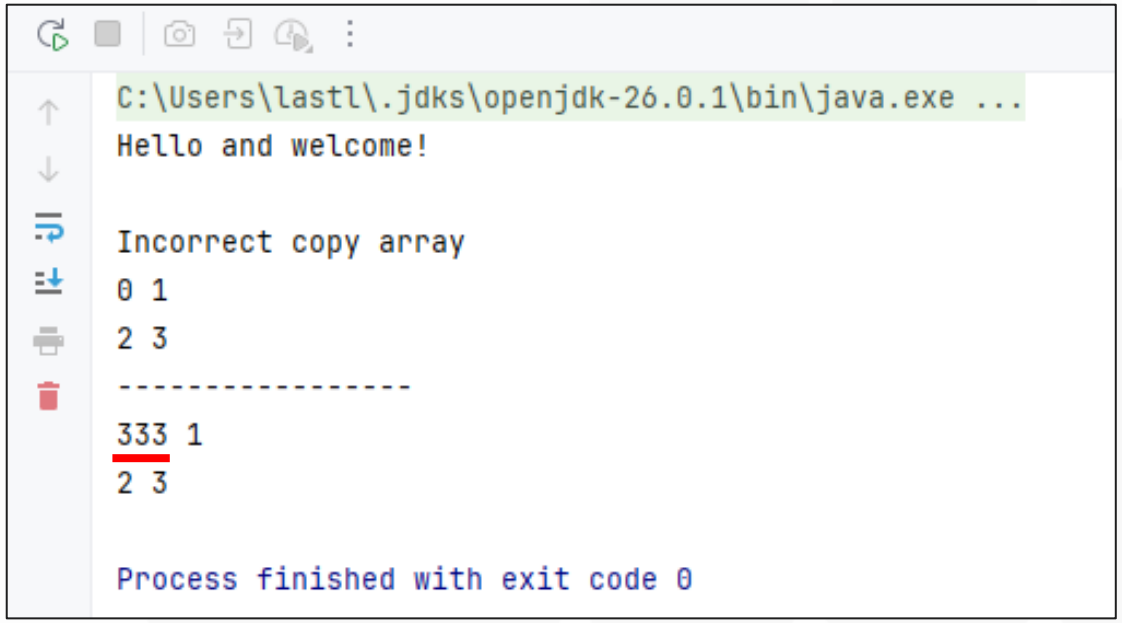
```
int size = 2;
int[][] source = {
        {0, 1},
        {2, 3}
};
int[][] dest = new int[size][size];

dest = source; // !!!!!!

System.out.println("-----------------");

source[0][0] = 333;
printArray(dest, size);
```

```
private void printArray(int[][] array, int size) {
      for (int i=0; i<size; i++) {
          for (int j=0; j<size; j++) {
              System.out.print (array[i][j] + " ");
          }
          System.out.println();
      }
  }
```



IThub

```java
public void copyArrayCorrectly() {
        int size = 2;
        int[][] source = {
                {0, 1},
                {2, 3}
        };
        int[][] dest = new int[size][size];

        for (int i=0; i<size; i++) {
            for (int j=0; j<size; j++) {
                dest[i][j] = source[i][j];
            }
        }

        printArray(dest, size);

        source[0][0] = 555;
        System.out.println("-----------------");
        printArray(dest, size);
    }
```



- Уже написанный метод System.arraycopy

IThub

IThub

Длина массива

Важнейшее свойство, которым обладают массивы, является свойство **length**, возвращающее **длину массива**, то есть количество его элементов:

```
int[] nums = {1, 2, 3, 4, 5};
int length = nums.length; // 5
```

Нередко бывает неизвестным последний индекс, и чтобы получить последний элемент массива, мы можем использовать это свойство:

```
int last = nums[nums.length-1];
```

IThub

Задание

Создайте программу, которая анализирует оценки группы студентов.

У вас есть массив оценок: `int[] grades = {5, 4, 3, 5, 2, 4, 5, 3};`

1. Вывести все оценки студентов

Программа должна вывести все элементы массива:

2. Найти среднюю оценку

Посчитать среднее арифметическое всех оценок.

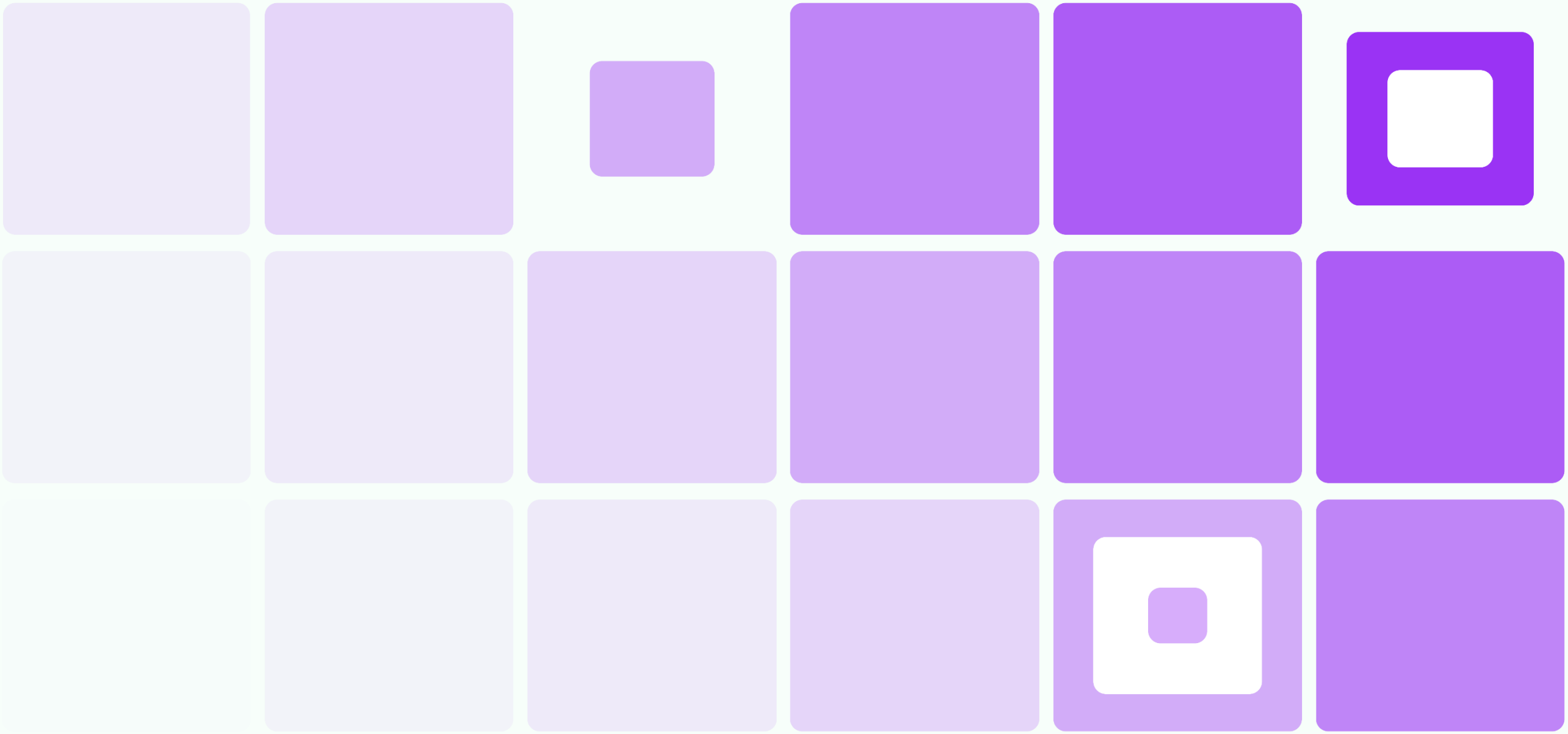
3. Найти лучшую оценку

Определить максимальную оценку в массиве.

4. Посчитать количество студентов с оценкой 5

5. Найти количество неудовлетворительных оценок

Посчитать, сколько студентов получили оценку меньше 3.



Методы

Методы

```
тип имя-метода(список параметров) {
    // тело метода
}
```

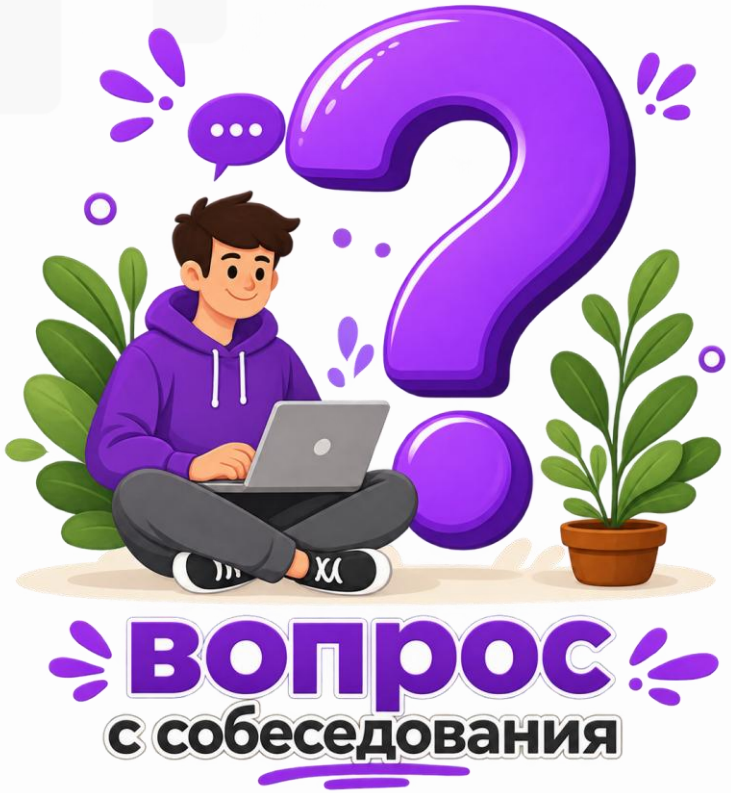
- тип = тип данных, возвращаемых методом. Если метод не возвращает значение, его возвращаемый тип void
- имя метода – любой допустимый идентификатор кроме тех, которые уже используются другими элементами в текущей области видимости
- список параметров представляет собой последовательность пар типов и идентификаторов, разделенных запятыми

```
public void setDepth(int depth) {
    this.depth = (double) depth;
}

public double getVolume() {
    return width * height * depth;
}

public void print() {
    System.out.println("Width: " + width);
    System.out.println("Height: " + height);
    System.out.println("Depth: " + depth);
}
```

IThub



Каким образом передаются переменные в методы, по значению или по ссылке?

В Java параметры всегда передаются только по значению, что определяется как «скопировать значение и передать копию». С примитивами это будет копия содержимого. Со ссылками - тоже копия содержимого, т.е. копия ссылки. При этом внутренние члены ссылочных типов через такую копию изменить возможно, а вот саму ссылку, указывающую на экземпляр - нет.

Функции

- функции должны быть компактными
- функции должны быть еще компактнее
- функция должна выполнять только одну операцию. Она должна выполнять ее хорошо, и ничего другого она делать не должна
- код должен читаться как рассказ – сверху вниз. За каждой функцией должны следовать функции следующего уровня абстракции
- используйте содержательные имена для функций. Не бойтесь использовать длинные имена. Не бойтесь расходовать время на выбор имени



IThub

- необходимость функций с большим количеством аргументов должна быть подкреплена очень вескими доводами — и все равно такие функции лучше не использовать
- аргументы-флаги (boolean) уродливы
- если функция должна получать более двух параметров, вероятно имеет смысл упаковать некоторые из аргументов в отдельных класс
- используйте исключения вместо возвращения кодов ошибок
- не возвращайте и не передавайте в вызове функции значение null. Это заставляет делать дополнительные проверки в вызывающем коде. Вместо null лучше вернуть исключение либо объект паттерна «особый случай»

ОПЕРАТОРЫ ПЕРЕХОДА

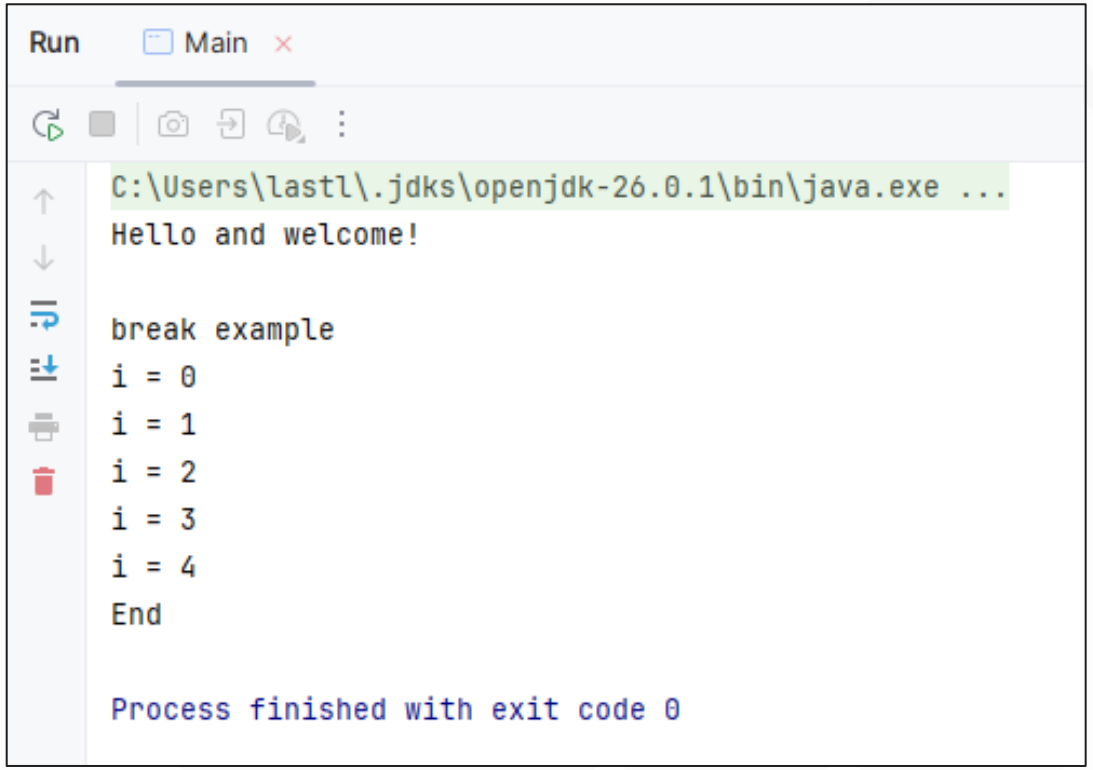
IThub

Оператор break

- завершает последовательность операторов в операторе switch
- можно использовать для выхода из цикла

Выход из цикла

```
public void breakOperatorExample() {
    System.out.println("break example");
    for (int i = 0; i < 100; i++) {
        if (i == 5) {
            break;
        }
        System.out.println("i = " + i);
    }
    System.out.println("End");
}
```



Оператор continue

- продолжает выполнение цикла, но прекращает обработку остатка кода в его теле для данной частной итерации

цикл for

```
public void continueOperatorForExample() {
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 0) continue;
        System.out.println("i = " + i);
    }
}
```



IThub

цикл while

```
public void continueOperatorWhileExample() {
    int i = -1;
    while (i < 10) {
        i++;
        if (i % 2 == 0) continue;
        System.out.println("i = " + i);
    }
}
```



Параметры переменной длины

Метод может принимать параметры переменной длины одного типа. Например, нам надо передать в метод набор чисел и вычислить их сумму, но мы точно не знаем, сколько именно чисел будет передано - 3, 4, 5 или больше. Параметры переменной длины позволяют решить эту задачу.

Троеточие перед названием параметра int ...nums указывает на то, что он будет необязательным и будет представлять массив. Мы можем передать в метод sum одно число, несколько чисел, а можем вообще не передавать никаких параметров. Причем, если мы хотим передать несколько параметров, то необязательный параметр должен указываться в конце.

```
public class Program {
    public static void main (String args[]) {
        sum(1, 2, 3); // 6
        sum(1, 2, 3, 4, 5); // 15
        sum(); // 0
     }

    static void sum(int ...nums) {
        int result = 0;
        for(int n: nums) {
            result += n;
         }
      System.out.println(result);
    }
}
```

Задание

Создайте программу, которая анализирует количество очков игроков турнира.

Есть массив результатов игроков:

int[] scores = {120, 250, 0, 450, 300, 500, 100};

1. Метод вывода результатов

Создайте метод: `void showScores(int[] scores)`

Метод должен вывести результаты всех игроков из массива.

2. Метод поиска (break)

Создайте метод: `boolean findScore (int[] scores, int target)`

Метод должен искать определённое количество очков.

- Если значение найдено: вывести сообщение: Результат найден: 500 остановить поиск с помощью break; вернуть true.
- Если значение отсутствует: Результат не найден и метод возвращает false.

3. Метод вывода активных игроков (continue)

Создайте метод: void showActivePlayers (int[] scores)

Условия: если игрок набрал 0 очков — пропустить его с помощью continue; вывести только игроков, которые набрали очки.